SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COURSE NAME: Theory of Computation COURSE CODE: CSA1301

COURSE OUTCOME COVERED

CO2: De**ign Finite Automata DFA, NFA, ε-NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)**

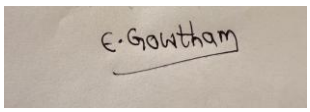
Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks:50
Duration : 1 Hour

Scenario Based

Code of Conduct:

I Enamala Gowtham / 192311190 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

1. A company is developing a smart security system for a research laboratory. Every employee entering the lab uses a digital access card that generates a sequence of symbols:

- a → Valid ID scan
- b → Biometric verification
- c → Door access confirmation

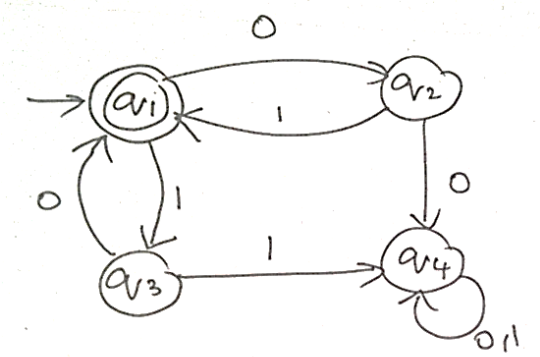
The security rule states that:

1. Every valid access sequence must start with a.
2. It may contain any number of biometric checks (b).
3. The sequence must end with c.

Examples of valid sequences: ac, abc, abbc, abbbc

2.Given a DFA used for validating a specific PIN pattern bbbc, bca ,c, ca, caaaaaa minimize the DFA and explain how state reduction improves performance and efficiency in embedded systems

3.Demonstrate how a given finite automaton can be converted into a regular expression



4.Consider the following grammar

S->aSc|T|U|e

T-> bTc|e

U->aUb|e

A} What is the language defined by the grammar?

B)Give the derivation tree for the word aaabbc

C Grammar is Ambiguous if there exists two distinct derivations for a word. Is the above grammar ambiguous? Justify.

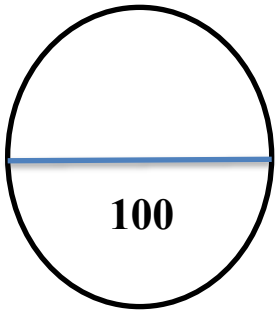
5.I)Show that if L1 and L2 are regular then (L1+l2) also regular

Ii)Is the language { a^p| p is prime} regular? Justify

SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RUBRICS FOR CALCULATION

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Q4 (10)	Q5 (10)	Total Marks (50)
Conceptual Understanding	30	3	3	3	3	3	15
Accuracy of Answer	25	2.5	2.5	2.5	2.5	2.5	12.5
Application of Knowledge	20	2	2	2	2	2	10
Completeness of Response	15	1.5	1.5	1.5	1.5	1.5	7.5
Clarity & Presentation	10	1	1	1	1	1	5
Total per Question	10 Marks	/10	/10	/10	/10	/10	/ 50



CO2 ASSESSMENT 3

Name : E.Gowtham

Reg.No : 192311190

Course : Theory of computation

Code : CSA1301

Q1. Design a DFA for the smart security system

Introduction

A Deterministic finite Automaton (DFA) is a mathematical model used to recognize patterns in input strings. for each state and input symbol, there is exactly one next state.

The smart security system uses three symbols:

- a - valid ID scan
- b - Biometric verification
- c - Door access confirmation

conditions

- The sequence must begin with a
- zero or more b symbols may occur after a
- The sequence must end with c
- NO symbols are allowed after c

Examples of accepted strings are ac, abc, abbc, abbbc.

Regular expression

$$L = ab^*c$$

Here, b^* represents 0 or more occurences of b.

State Definition

State	Description
q_0	Initial state
q_1	Valid ID received
q_2	Accepting state
q_d	Dead state

Transition table

State	a	b	c
$\rightarrow q_0$	q_1	q_d	q_d
q_1	q_d	q_1	q_2
$*q_2$	q_d	q_d	q_d
q_d	q_d	q_d	q_d

$$q_0 \xrightarrow{a} q_1 \xrightarrow{b} q_1 \xrightarrow{b} q_1 \xrightarrow{c} q_2$$

The automation ends at q_2, so the string is accepted.

Applications

- Pattern recognition
- Access-card validation

Conclusion: The DFA accepts only strings of the form ab^*c, ensuring that the access sequence follows the required security rules.

Minimize the DFA for the given PIN patterns

DFA minimization is the process of reducing the number of states in a deterministic finite automation without changing the language it accepts. The minimized DFA is equivalent to the original DFA because both accept and reject the same strings

Steps involved in DFA minimization

- Identify the states
- Remove unreachable states
- Divide the states into groups
- check state equivalence
- Refine the groups
- Merge equivalent states

Example of state equivalence

Suppose two states q1 and q2 are both non-final, and for every input symbol they tran-sition to the same respective states or to equivalent states. Then q1 and q2 can be merged. However, if a leds from q1 to a final state and from q2 to a non-final

state, they are distinguishable and cannot be merged.

Advantages of DFA minimization

- Reduces the number of states
- Reduces memory required for storing the transition table
- Simplifies the implementation of control logic
- Can reduce processing overhead in input validation
- Preserve the original language and validation behavior

Applications

DFA minimization is useful in digital PIN validation, smart cards, lexical analyzer, communication protocols.

Conclusion

DFA minimization remove redundant states while preserving the accepted language. To obtain the exact minimized DFA for the listed PIN patterns, transition table is required.

Demonstrate how a finite automaton can be converted into a regular expression

Introduction

A finite automaton is a computational model that accepts or rejects strings. A regular expression is a symbolic representation of a regular language. Both represent regular language, so an equivalent regular expression can be obtained from any time automaton using the state elimination method.

operator	example
Union (U)	a∪b
Concatentation	ab
Kleene starc*)	a*b
Empty string (ε)	ε

State elimination method

- Identify the initial state and all final state
- Add a new initial state with an ε-transition to the original initial state.

- Add a new final state and connect every original final state to it using an ε-trans
- choose an intermediate state for elimin
- update the transition labels to account for all paths through the state
- Repeat until only the new initial and final states remain
- The resulting transition label is the required regular expression

Applications

- Compiler lexical analysis
- Token recognition
- Input validation
- Text searching and pattern matching

Conclusion

The state elimination method produces a regular expression equivalent to the given finite automation. the example demonstr-ates the method;

Consider the grammar

Given grammar:

$S \rightarrow aSc \mid T \mid U \mid \varepsilon$

$T \rightarrow bTc \mid \varepsilon$

$U \rightarrow aUb \mid \varepsilon$

The language defined by the grammar

A context-free grammar consists of terminals, non-terminals, production rules and a start symbol.

Start Symbol: S

Terminals: a, b, c

Non-Terminals: S, T, U

Empty string: ε

$S \rightarrow aSc$

$T \rightarrow bTc \mid \varepsilon$

Generates equal numbers of b and c:

ε, bc, bbcc, bbbccc, ...

$U \rightarrow aUb \mid \varepsilon$

Generates equal numbers of a and b:

ε, ab, aabb, aaabbb, ...

The last two steps can be written more clea
by applying $U \to aUb$ twice and then U-

$S \Rightarrow aSC \Rightarrow aUc \Rightarrow aaUbc \Rightarrow aaa(aUb)bc$.

The corresponding production sequence is $S \to a$
$S \to U$, followed by two applications of $U \to aU$
and finally $U \to \varepsilon$

A grammar is ambiguos if atleast one string
has two or more distinct derivations or
parse tree

Consider the empty string ε

$S \Rightarrow \varepsilon$

$S \Rightarrow T \Rightarrow \varepsilon$

$S \Rightarrow U \Rightarrow \varepsilon$

The same string has multiple distinct
derivations.

Result: The grammar is ambiguous beca-
-use the empty string has more than
one derivation.

show that if L_1 and L_2 are regular, then : $L_1 \cup L_2$ is also regular

A language is regular if it can be accepted by a finite automaton or represented by a regular expression.

Proof

Let L_1 and L_2 are 2 regular language

There exist finite automata M_1 and M_2 than accept them.

construct a new finite automaton that simulates both automata in parallel. Its states are pairs of states from M_1 and M_2. A pair is accepting if either component is an accepting state of its respective automaton.

$$L(M) = L_1 \cup L_2$$

Since the product automaton has a finite number of states, it is a finite automaton.

Is the language $\{a^p \mid p \text{ is prime}\}$ regular? Justify

No, the language is not regular.

Let

$L = \{a^p \mid p \text{ is prime}\}$

Assume that L is regular. By the pumping lemma, there exists a pumping length n.

Prime number $> n$ and the string $w = a^p$

According to the pumping lemma, $w = xyz$

- $|xy| \leq n$
- $|y| \geq 0$
- $xy^iz \in L$ for every integer $i \geq 0$

Conclusion, the language cannot be recognized by any finite automata.